

IMAGES ARE ARRAY [PART 2: DICOM]

Nada Fitrieyatul Hikmah

Loading Images (DICOM)

- **imageio** : read and save images.

To install imageio, use one of the following methods:

- If you are in a conda env: `conda install -c conda-forge imageio`
- If you have pip: `pip install imageio`
- Good old `python setup.py install`

```
In [1]: 1 import imageio
        2
        3 im = imageio.imread('D:\Teaching\Pengolahan Citra Medika\Database\IM-0001-0001.dcm')
        4 type(im)
```

```
Out[1]: imageio.core.util.Array
```

```
In [2]: 1 im
```

```
Out[2]: Array([[ -991, -1011, -1024, ..., -975, -955, -966],
               [ -998, -1007, -1024, ..., -971, -966, -977],
               [-1000, -1005, -1021, ..., -974, -975, -968],
               ...,
               [ -833,  -804,  -753, ..., -331, -377, -386],
               [ -912,  -885,  -846, ..., -540, -584, -591],
               [ -947,  -939,  -934, ..., -685, -723, -736]], dtype=int16)
```

Loading Images

- Slice the array by specifying values along each available dimension.

```
In [6]: 1 im[0,0]
```

```
Out[6]: -991
```

```
In [7]: 1 im[0:2,0:2]
```

```
Out[7]: Array([[ -991, -1011],  
               [ -998, -1007]], dtype=int16)
```

Metadata

- Images are always acquired in a specific context. This information is often referred to as **metadata**.
- Accessible in image objects through the **meta** dictionary attribute.

```
In [3]: 1 im.meta
```

```
Out[3]: Dict([('TransferSyntaxUID', '1.2.840.10008.1.2.1'),  
             ('SOPClassUID', '1.2.840.10008.5.1.4.1.1.2'),  
             ('SOPInstanceUID',  
              '1.3.12.2.1107.5.1.4.50025.30000005060811542834300000777'),  
             ('StudyDate', '20050601'),  
             ('SeriesDate', '20050601'),  
             ('AcquisitionDate', '20050608'),  
             ('ContentDate', '20050608'),  
             ('StudyTime', '120000.000000 '),  
             ('SeriesTime', '120000.000000 '),  
             ('AcquisitionTime', '103434.945159 '),  
             ('ContentTime', '103434.945159 '),  
             ('Modality', 'CT'),
```

```
In [4]: 1 im.meta['Modality']
```

```
Out[4]: 'CT'
```

```
In [5]: 1 im.meta.keys()
```

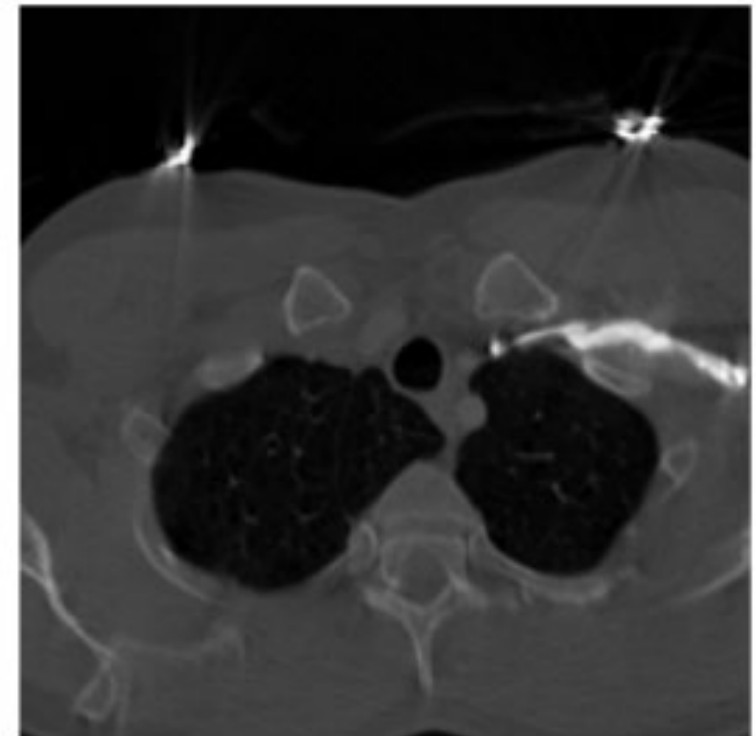
```
Out[5]: odict_keys(['TransferSyntaxUID', 'SOPClassUID', 'SOPInstanceUID', 'StudyDate',  
                   'SeriesDate', 'AcquisitionDate', 'ContentDate', 'StudyTime', 'SeriesTime', 'Acq  
u  
isitionTime', 'ContentTime', 'Modality', 'Manufacturer', 'InstitutionName', 'S  
tudyDescription', 'SeriesDescription', 'PatientName', 'PatientID', 'PatientBirt  
hDate', 'PatientSex', 'PatientAge', 'StudyInstanceUID', 'SeriesInstanceUID', 'S  
eriesNumber', 'AcquisitionNumber', 'InstanceNumber', 'ImagePositionPatient', 'I  
mageOrientationPatient', 'SamplesPerPixel', 'Rows', 'Columns', 'PixelSpacing',  
'BitsAllocated', 'BitsStored', 'HighBit', 'PixelRepresentation', 'RescaleInterc  
ept', 'RescaleSlope', 'PixelData', 'shape', 'sampling'])
```

Plotting Images

- Matplotlib's `imshow()` function displays 2D image data.
- Many colormaps available but often shown in grayscale (`cmap='gray'`)
- Axis ticks and labels are often not useful for images

In [12]:

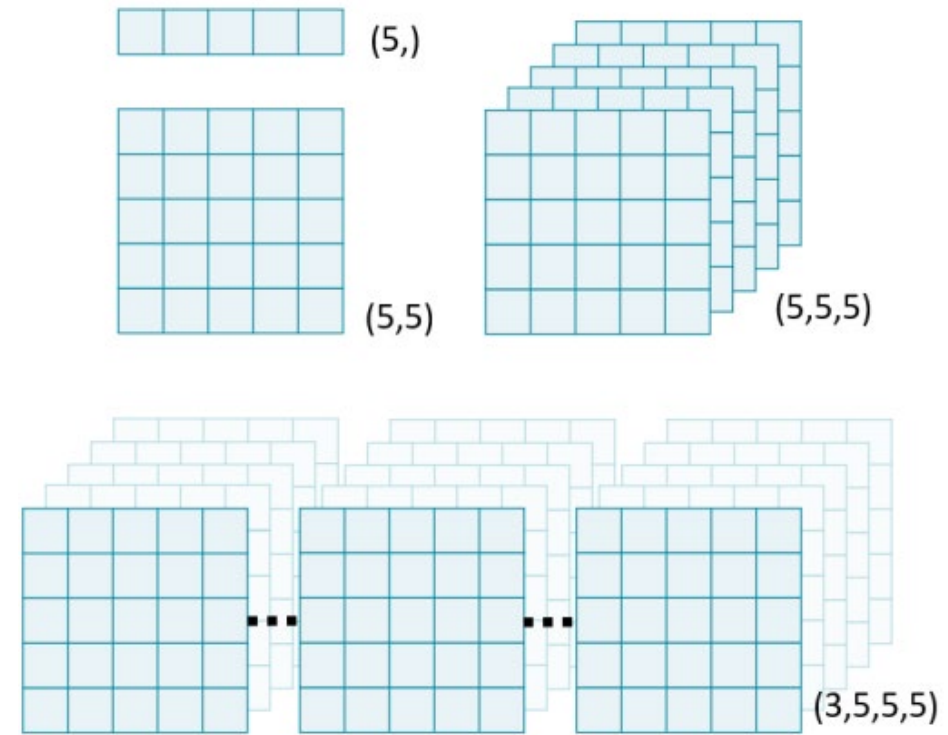
```
1 import matplotlib.pyplot as plt
2 plt.imshow(im,cmap='gray')
3 plt.axis('off')
4 plt.show()
```





N-DIMENSIONAL IMAGES

N-dimensional images are stacks of arrays



```
In [30]: 1 import imageio
          2 import numpy as np
          3 im1=imageio.imread('D:\Teaching\Pengolahan Citra Medika\Database\dataset1_Thorax_Abdomen\IM-0001-0015.dcm')
          4 im2=imageio.imread('D:\Teaching\Pengolahan Citra Medika\Database\dataset1_Thorax_Abdomen\IM-0001-0016.dcm')
          5 im3=imageio.imread('D:\Teaching\Pengolahan Citra Medika\Database\dataset1_Thorax_Abdomen\IM-0001-0017.dcm')
          6
          7 im1.shape
```

Out[30]: (512, 512)

```
In [51]: 1 vol = np.dstack([im1,im2,im3])
          2 vol.shape
```

Out[51]: (512, 512, 3)

Loading volumes directly

Imageio.volread():

- Read multi-dimensional data directly.
- Assemble a volume from multiple images

```
1 import os
2 os.listdir('D:\\Teaching\\Pengolahan Citra Medika\\Databa
3 'IM-0001-0016.dcm',
4 'IM-0001-0017.dcm',
5 'IM-0001-0018.dcm',
6 'IM-0001-0019.dcm',
7 'IM-0001-0020.dcm',
8 'IM-0001-0021.dcm',
9 'IM-0001-0022.dcm',
10 'IM-0001-0023.dcm',
11 'IM-0001-0024.dcm',
12 'IM-0001-0025.dcm',
13 'IM-0001-0026.dcm',
14 'IM-0001-0027.dcm',
15 'IM-0001-0028.dcm',
16 'IM-0001-0029.dcm',
17 'IM-0001-0030.dcm',
18 'IM-0001-0031.dcm',
19 'IM-0001-0032.dcm',
20 'IM-0001-0033.dcm',
21 'IM-0001-0034.dcm',
22 'IM-0001-0035.dcm'.
```

```
In [60]: 1 import imageio
2 vol1 = imageio.volread('D:\\Teaching\\Pengolahan Citra Medika\\Database\\dataset1_Thorax_Abdomen')
3 vol1.shape
```

Reading DICOM (examining files): 1/291 files (0.392/291 files (31.6189/291 files (64.9%233/291 files (80.1%291/291 files (100.0%)

Found 1 correct series.

Reading DICOM (loading data): 37/291 (12.7%79/291 (27.1%121/291 (41.6161/291 (55.3205/291 (70.4240/291 (82.5274/291 (94.2291/291 (100.0%)

```
Out[60]: (291, 512, 512)
```


Sampling and field of view

- **Sampling rate:** physical space covered by each element.
- **Field of view:** physical space covered along each axis.

```
1 #sampling rate (in mm)
2 d0, d1, d2 = vol1.meta['sampling']
3 d0, d1, d2 #d0 = jarak antar slicing,
```

(2.0, 0.5078125, 0.5078125)

```
1 #Field of view (in mm)
2 n0 * d0, n1 * d1, n2 * d2
```

(582.0, 260.0, 260.0)

A vertical bar on the left side of the slide, transitioning from orange at the top to purple at the bottom.

IMAGE PLOTTING

Plotting multiple images at once

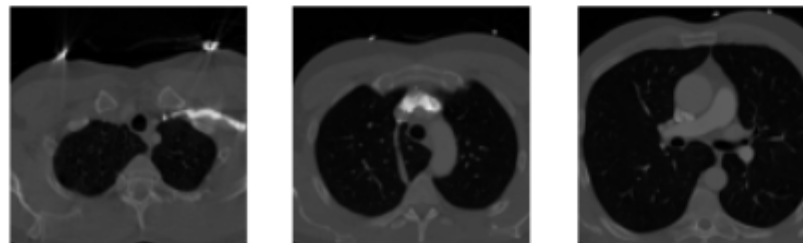
- `plt.subplots`: creates a figure canvas with multiple **AxesSubplots** objects.

```
1 import imageio
2 vol2 = imageio.volread('D:\Teaching\Pengolahan Citra Medika\Database\dataset
3 fig, axes = plt.subplots(nrows=1,ncols=3)
4
5 axes[0].imshow(vol[0],cmap='gray')
6 axes[1].imshow(vol[20],cmap='gray')
7 axes[2].imshow(vol[50],cmap='gray')
8
9 for ax in axes:
10     ax.axis('off')
11
12 plt.show()
```

Reading DICOM (examining files): 1/291 files (0.3240/291 files (82.5%291/291 files (100.0%))

Found 1 correct series.

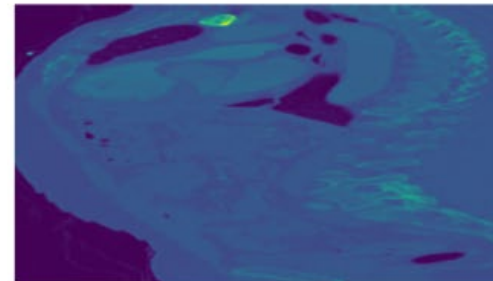
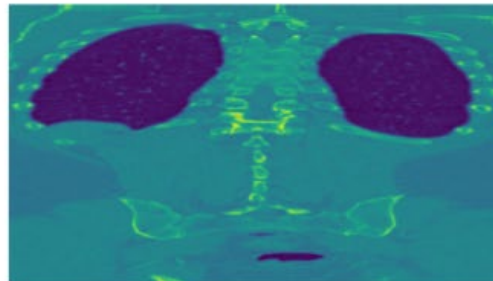
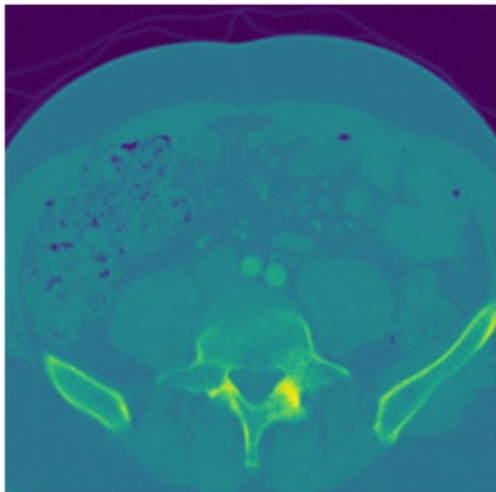
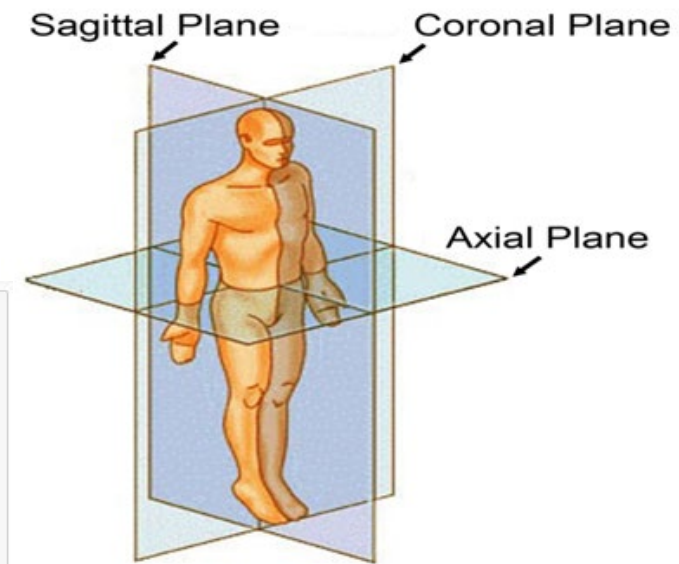
Reading DICOM (loading data): 96/291 (33.0%200/291 (68.7291/291 (100.0%))



Another views

```
1 import imageio
2 vol3 = imageio.volread('D:\Teaching\Pengolahan Citra Medika\Database\dataset1_Thorax_Abdomen')
3 fig, axes = plt.subplots(nrows=1,ncols=3, figsize=(20,10))
4 view_axial = vol3[200, :, :]
5 view_coronal = vol3[:, 450, :]
6 view_sagittal = vol3[:, :, 300]
7
8 axes[0].imshow(view_axial)
9 axes[1].imshow(view_coronal)
10 axes[2].imshow(view_sagittal)
11
12 for ax in axes:
13     ax.axis('off')
14
15 plt.show()
16
```

Reading DICOM (examining files): 1/291 files (0.3226/291 files (77.7%291/291 files (100.0%))
Found 1 correct series.
Reading DICOM (loading data): 97/291 (33.3%190/291 (65.3286/291 (98.3291/291 (100.0%))



Modifying the aspect ratio (1)

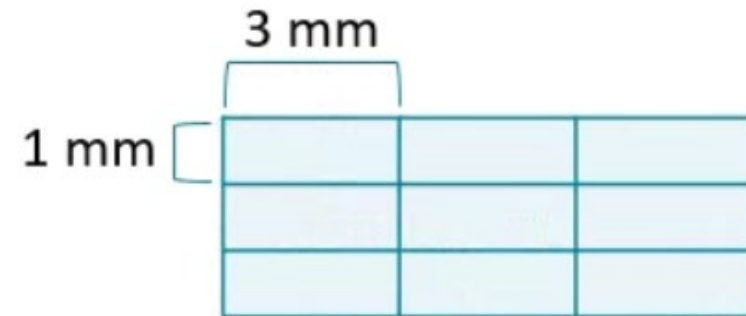
- Many datasets do not have equal sampling rates across all dimensions. In these cases, we will want to **stretch** the pixels along one side to account for the differences.
- Any two dimensions of an array can form an image, and slicing along different axes can provide a useful perspective.
- However, unequal sampling rates can create distorted images.
- Changing the **aspect ratio** can address this by increasing the width of one of the dimensions.

Pixels may adopt any aspect ratio:

4:1

16:9

1:1



Shape: (3, 3), Sampling: (1, 3)

Modifying the aspect ratio (2)

- This results in a properly proportioned image.
- Failing to adjust the aspect would have resulted in a distorted image.

```
In [121]: 1 view_sagittal = vol3[:, :, 300]
          2 d0, d1, d2 = vol3.meta['sampling']
          3 d0, d1, d2
```

```
Out[121]: (2.0, 0.5078125, 0.5078125)
```

```
In [122]: 1 asp1 = d0/d1
          2 asp1
```

```
Out[122]: 3.9384615384615387
```

```
In [124]: 1 plt.imshow(view_sagittal, aspect = asp1)
          2 plt.axis('off')
          3 plt.show()
```



A vertical bar on the left side of the slide, transitioning from orange at the top to purple at the bottom.

OTHER WAYS...

Handle DICOM Files (1)

References :

(1) Albertina, B., Watson, M., Holback, C., Jarosz, R., Kirk, S., Lee, Y., ... Lemmerman, J. (2016). Radiology Data from The Cancer Genome Atlas Lung Adenocarcinoma [TCGA-LUAD] collection. The Cancer Imaging Archive. <http://doi.org/10.7937/K9/TCIA.2016.JGNIHEP5>

(2) Clark K, Vendt B, Smith K, Freymann J, Kirby J, Koppel P, Moore S, Phillips S, Maffitt D, Pringle M, Tarbox L, Prior F. The Cancer Imaging Archive (TCIA): Maintaining and Operating a Public Information Repository, Journal of Digital Imaging, Volume 26, Number 6, December, 2013, pp 1045-1057.

Handle DICOM Files (2)

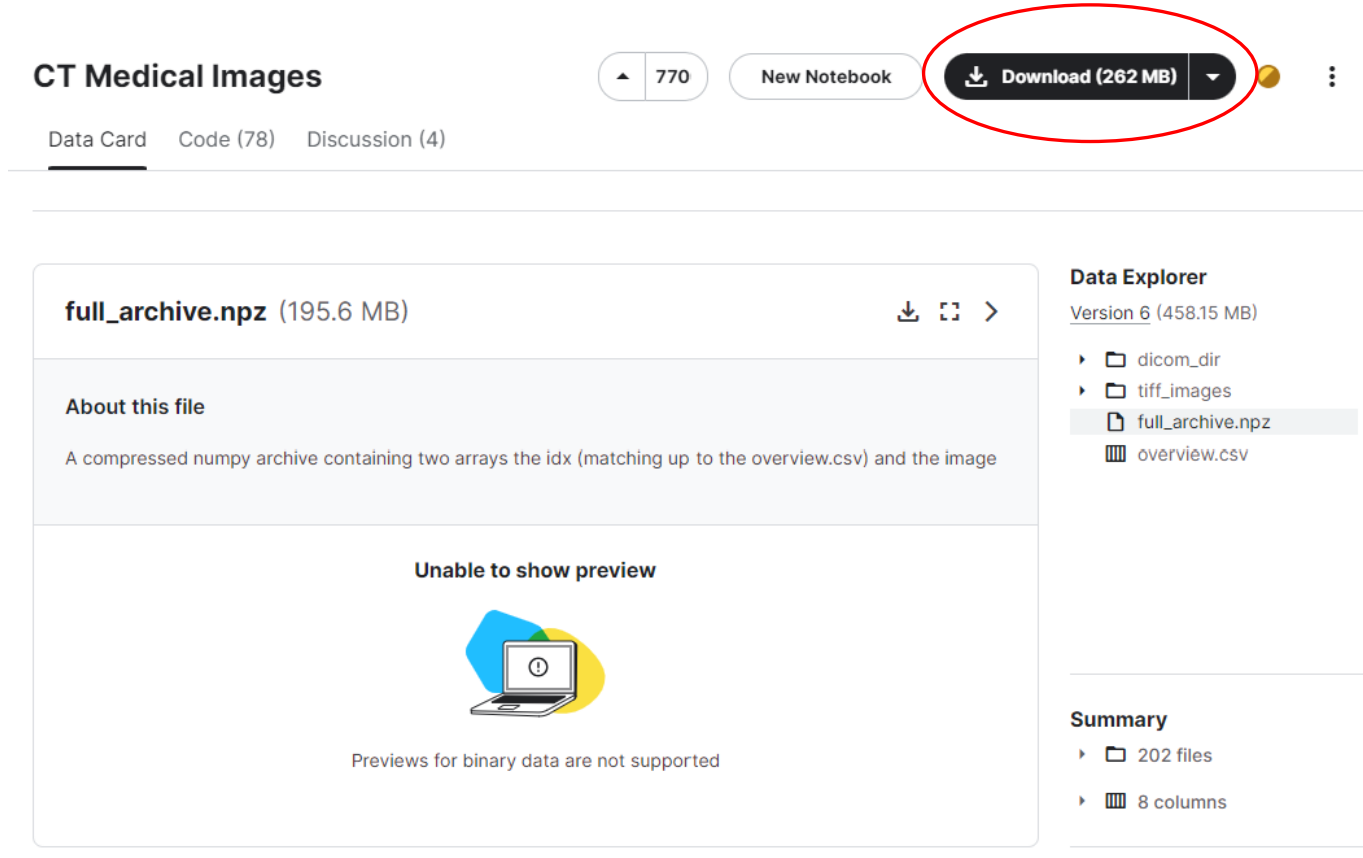
Download data:

❑ Log in to Kaggle

❑ Open link :

https://www.kaggle.com/kmader/siim-medical-images?select=dicom_dir

❑ Click on Download



CT Medical Images 770 New Notebook **Download (262 MB)**

Data Card Code (78) Discussion (4)

full_archive.npz (195.6 MB) Download Full Screen Expand

About this file

A compressed numpy archive containing two arrays the idx (matching up to the overview.csv) and the image

Unable to show preview

Previews for binary data are not supported

Data Explorer
Version 6 (458.15 MB)

- dicom_dir
- tiff_images
- full_archive.npz**
- overview.csv

Summary

- 202 files
- 8 columns

Imports

```
from pathlib import Path

import pydicom
import matplotlib.pyplot as plt
import numpy as np
```

- ❑ pathlib for easy path handling
- ❑ pydicom to handle dicom files
- ❑ matplotlib for visualization
- ❑ numpy to create the 3D container

Read a single dcm file

```
sample_dcm = "ID_0000_AGE_0060_CONTRAST_1_CT.dcm"  
dicom_file = pydicom.read_file(sample_dcm) # read single dicom file
```

```
print(dicom_file)
```

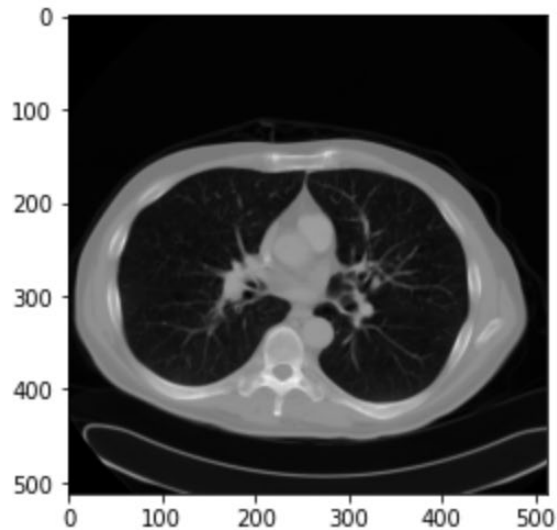
```
Dataset.file_meta -----  
(0002, 0000) File Meta Information Group Length  UL: 194  
(0002, 0001) File Meta Information Version       OB: b'\x00\x01'  
(0002, 0002) Media Storage SOP Class UID        UI: CT Image Storage  
(0002, 0003) Media Storage SOP Instance UID      UI: 1.3.6.1.4.1.14519.5.2.1.7777.9002.108821836759549281694712274169  
(0002, 0010) Transfer Syntax UID                UI: Implicit VR Little Endian  
(0002, 0012) Implementation Class UID           UI: 1.2.40.0.13.1.1.1  
(0002, 0013) Implementation Version Name        SH: 'dcm4che-1.4.31'  
-----  
(0008, 0000) Group Length                       UL: 430  
(0008, 0005) Specific Character Set              CS: 'ISO_IR 100'  
(0008, 0008) Image Type                         CS: ['ORIGINAL', 'PRIMARY', 'AXIAL']  
(0008, 0016) SOP Class UID                      UI: CT Image Storage  
(0008, 0018) SOP Instance UID                   UI: 1.3.6.1.4.1.14519.5.2.1.7777.9002.108821836759549281694712274169  
(0008, 0020) Study Date                         DA: '19830708'  
(0008, 0021) Series Date                       DA: '19830708'  
(0008, 0022) Acquisition Date                  DA: '19830708'  
(0008, 0023) Content Date                      DA: '19830708'  
(0008, 0030) Study Time                        TM: '085723.000000'  
(0008, 0031) Study Time                        TM: '19830708.000000'
```

All information
about dicom file can
be accessed.

Show dicom image

```
ct = dicom_file.pixel_array # Load the image pixel data as a numpy array  
plt.figure()  
plt.imshow(ct, cmap="gray")
```

<matplotlib.image.AxesImage at 0x27acdb01ee0>



Get information about shape of image :

```
print(ct.shape)
```

(512, 512)

3D Data

□ Data of full head MRI scan :
<https://zenodo.org/record/16956#.YFMM5PtKiV5>

□ Ref : Lionheart, W. R. B. (2015).
An MRI DICOM data set of the
head of a normal male human aged
52 [Data set].
Zenodo. <http://doi.org/10.5281/zenodo.16956>

Files

The screenshot shows a file explorer interface for a ZIP file named 'DICOM.zip'. The interface displays a hierarchical tree structure of folders and files, including .DS_Store, ST000000, and SE000000, with their respective sizes. At the bottom, a table lists the files, and a red circle highlights the 'Download' button.

Name	Size	Actions
DICOM.zip	4.8 MB	Preview Download

Make list of 3D Data (1)

```
path_to_head_mri = Path("SE000001")
```

Use the glob function to return all items in a directory which correspond to the provided pattern. As in this case, the directory only contains the DICOM files
➔ return all files in it ("*")

```
[WindowsPath('SE000001/MR000000'),  
WindowsPath('SE000001/MR000001'),  
WindowsPath('SE000001/MR000002'),  
WindowsPath('SE000001/MR000003'),  
WindowsPath('SE000001/MR000004'),  
WindowsPath('SE000001/MR000005'),  
WindowsPath('SE000001/MR000006'),  
WindowsPath('SE000001/MR000007'),  
WindowsPath('SE000001/MR000008'),  
WindowsPath('SE000001/MR000009'),  
WindowsPath('SE000001/MR000010'),  
WindowsPath('SE000001/MR000011'),  
WindowsPath('SE000001/MR000012'),  
WindowsPath('SE000001/MR000013'),  
WindowsPath('SE000001/MR000014'),  
WindowsPath('SE000001/MR000015'),  
WindowsPath('SE000001/MR000016'),  
WindowsPath('SE000001/MR000017'),  
WindowsPath('SE000001/MR000018'),  
WindowsPath('SE000001/MR000019'),  
WindowsPath('SE000001/MR000020'),  
WindowsPath('SE000001/MR000021'),  
WindowsPath('SE000001/MR000022'),  
WindowsPath('SE000001/MR000023'),  
WindowsPath('SE000001/MR000024'),  
WindowsPath('SE000001/MR000025'),  
WindowsPath('SE000001/MR000026')]
```

```
all_files = list(path_to_head_mri.glob("*")) # as glob returns a generator, we convert it to a list
```

```
all_files # make sure that all files are present in the list
```

Make list of 3D Data (2)

```
mri_data = []

for path in all_files:
    data = pydicom.read_file(path) # read the single DICOM files
    mri_data.append(data)
print(len(mri_data))
```

27

It is possible that the DICOM files are not ordered according to their actual image position → This can be verified by inspecting the **SliceLocation**.

```
for slice in mri_data[:5]:
    print(slice.SliceLocation)
```

```
89.9999955528687
107.999995419197
77.9999962275740
131.999997780749
149.999999502083
```

Make list of 3D Data (3)

It crucial to order them → use the **"SliceLocation"** attribute passed to the sorted function to identify the 2D slice position and thus order the slices.

```
# this sorts the slices according to their location
mri_data_ordered = sorted(mri_data, key=lambda slice: slice.SliceLocation)

### Ordered slices ###
for slice in mri_data_ordered[:5]:
    print(slice.SliceLocation)
```

```
0.0
5.99999663091323
11.9999973042441
17.9999979772582
23.9999946081714
```


Store 3D data in a list

Extract the actual data (pixel_arrays) from Dicome files and store in a list:

```
full_volume = []  
for slice in mri_data_ordered:  
    full_volume.append(slice.pixel_array) # fill the 3D array in a slice-per-slice manner  
full_volume = np.array(full_volume)  
print(full_volume.shape)
```

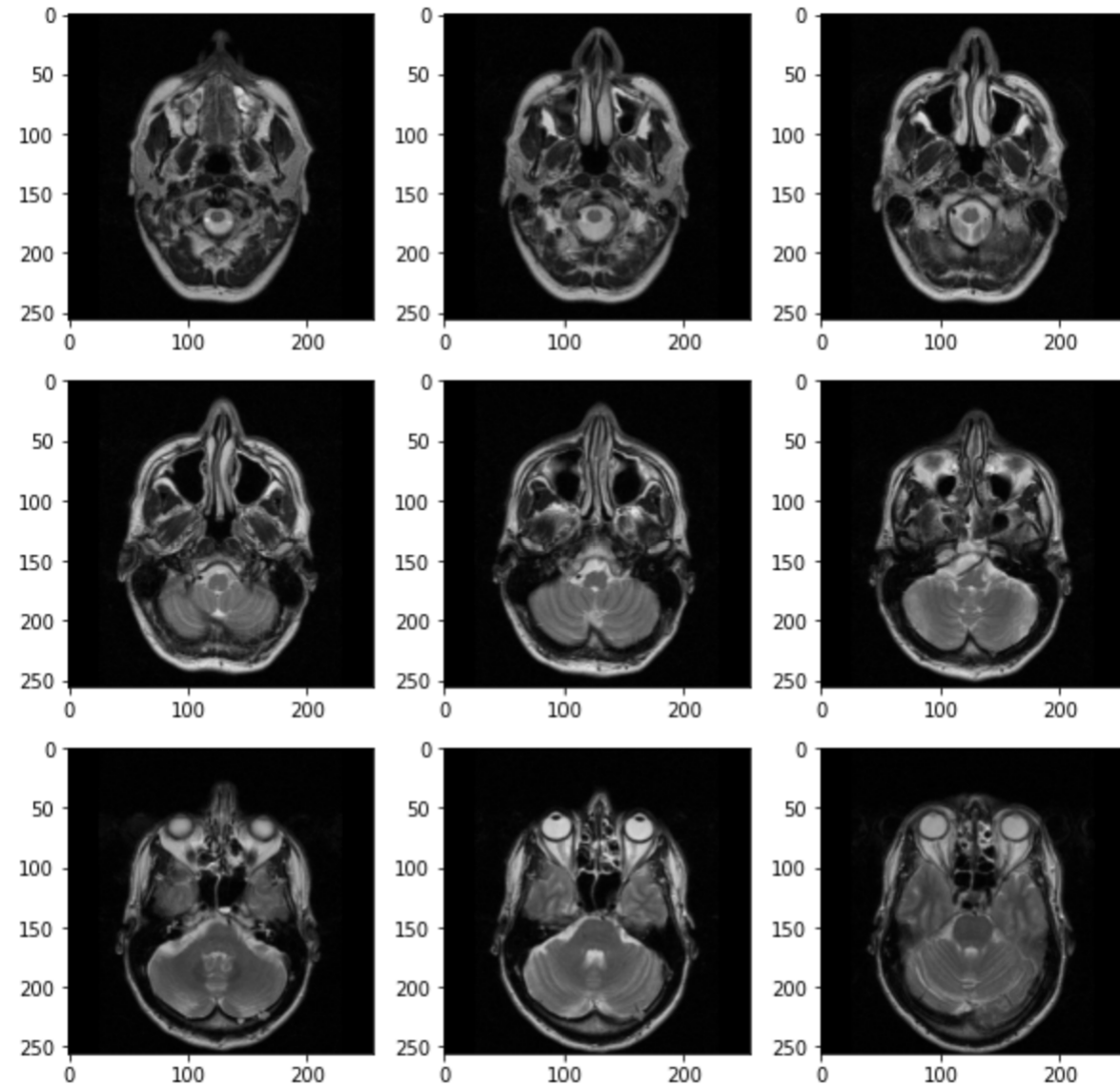
```
(27, 256, 256)
```

Show 3D data

Some slices of the ordered 3D volume:

```
fig, axis = plt.subplots(3, 3, figsize=(10, 10))

slice_counter = 0
for i in range(3):
    for j in range(3):
        axis[i][j].imshow(full_volume[slice_counter], cmap="gray")
        slice_counter+=1
```



NIFTI

(NEUROIMAGING INFORMATICS TECHNOLOGY
INITIATIVE)

What is NIfTI?

- ❑ An open file format for storage of medical image data (historically used for neuroimaging – hence the name, but not restricted to neuroimaging).
- ❑ Efficiently store medical image data together with necessary metadata.
- ❑ Mainly used in research settings.
- ❑ Not a clinical standard.

Structure

- ❑ Header : Containing information mainly about image geometry (resolution, position, orientation).
- ❑ Body : Actual image pixel data (2D, 3D, 4D, ...)
- ❑ In general, easier to handle than DICOM files.
- ❑ File extension usually “.nii” or “.nii-gz” (compressed version)

How to work with NIfTI files

- ❑ Python libraries : NiBabel, SimpleITK.
- ❑ Image viewer : 3D Slicer (www.slicer.org)
- ❑ Transform DICOM to NIfTI in Python ➔ dicom2nifti

Convert DICOM to NIfTI

Convert DICOM to NIfTI using last DICOM data :

```
import dicom2nifti  
from pathlib import Path
```

```
path_to_dicom = "SE000001"
```

```
dicom2nifti.convert_directory(path_to_dicom, ".")
```

- Generating file `201_t2w_tse.nii.gz` in the `path_to_nifti`.
- Creating complete 3D MRI Scan.

Read NIfTI files (1)

- ❑ nibabel to handle nifti files
- ❑ matplotlib to plot the brain images
- ❑ load using *nib.load(path)*

```
import nibabel as nib
import matplotlib.pyplot as plt
```

```
nifti = nib.load('201_t2w_tse.nii.gz')
```


Read NIfTI files

```
print(nifti)
```

```
<class 'nibabel.nifti1.Nifti1Image'>
data shape (256, 256, 27)
affine:
[[-9.36898887e-01  3.20514254e-02  6.37919828e-02  1.15272324e+02]
 [ 3.03588901e-02  9.27917123e-01 -8.33337128e-01 -9.72956390e+01]
 [ 1.43172191e-02  1.29802674e-01  5.94150448e+00 -8.23735046e+01]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.00000000e+00]]
metadata:
<class 'nibabel.nifti1.Nifti1Header'> object, endian='<'
sizeof_hdr      : 348
data_type       : b''
db_name         : b''
extents         : 0
session_error   : 0
regular         : b''
dim_info        : 0
dim             : [ 3 256 256 27  1  1  1  1]
intent_p1       : 0.0
intent_p2       : 0.0
intent_p3       : 0.0
intent_code     : none
datatype        : uint16
bitpix         : 16
slice_start     : 0
pixdim         : [-1.          0.9375      0.9375      5.9999995  1.          1.
 1.          1.          ]
vox_offset     : 0.0
scl_slope      : nan
scl_inter      : nan
slice_end      : 0
slice_code     : unknown
xyzt_units     : 2
cal_max        : 0.0
cal_min        : 0.0
```

Image Shape and Image Pixel Data

```
print(nifti.shape) # get the image shape
print(nifti.header.get_data_shape()) # get the image shape in a different way
```

```
(256, 256, 27)
(256, 256, 27)
```

Obtaining image pixel data → The image pixel data can be extracted using the *get_fdata()* function of the nifti object.

```
image_array = nifti.get_fdata()
print(image_array.dtype, image_array.shape)
```

```
float64 (256, 256, 27)
```

View the image

```
fig, axis = plt.subplots(3, 3, figsize=(10, 10))

slice_counter = 0
for i in range(3):
    for j in range(3):
        axis[i][j].imshow(image_array[:, :, slice_counter], cmap="gray")
        slice_counter+=1
```

